

Communicating between threads

Demo.kt

- some bugs to fix.

In Java/Kotlin, there are
"Spurious wakes."

- replace "if" with while

There's no guarantee some other
unexpected thread wasn't the
one that woke you.

So you should signal all threads.

- One of them will be happy to be
awake and proceed.

When awaiting, it releases the lock.
When it wakes, it then competes to
reacquire.

Conclusion: don't write low-level
code like this, use well-written
(built-in) libraries for communication

Locking algorithm

Typically, locking is given to us by the tools we're using.

That said, what might some locking algs look like?

Peterson lock / alg, for 2 threads
→ we're looking at.

Do this in steps.

Imagine we're creating our own Lock class

```
class Lock {  
  fun lock() {  
    ...  
  }  
  fun unlock() {  
    ...  
  }  
}
```

```
val l = Lock()  
l.lock()  
=====  
Critical section  
=====  
l.unlock()
```

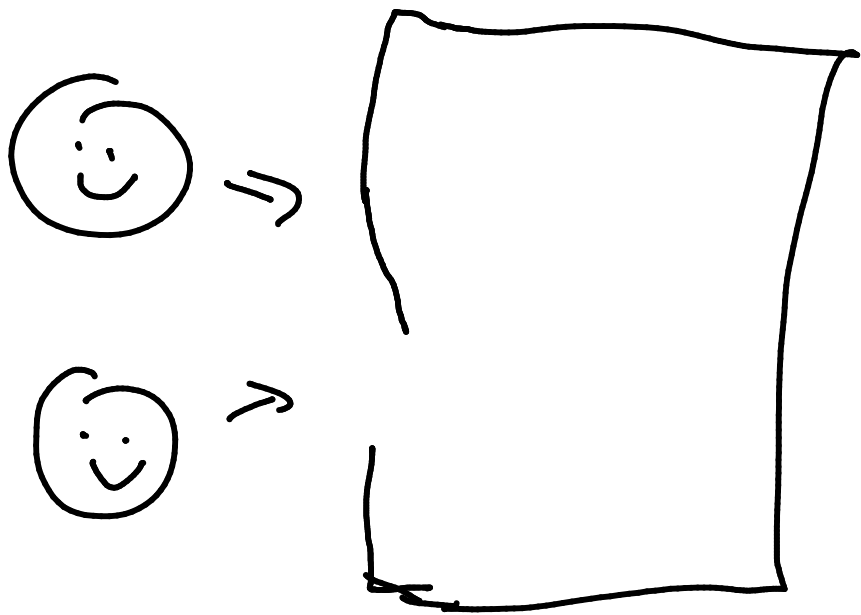
What are useful properties for a lock to have?

① Mutual exclusion - makes sure critical sections don't overlap

② ~~Self~~-deadlock - freedom
implicit

- it at least one thread wants the lock, one of will get it eventually (so long as someone else doesn't have it forever)

keep
bouncing
against
each other



③ Starvation - freedom - every thread that wants the lock eventually gets it (assuming no one holds it forever)

Assumption

- volatile variables are atomic
- only one thread accesses at a time
- no two lines of code in different threads occur precisely simultaneously

```
class Lock1 {  
    val flag = mutableListOf(false, false)
```

```
    fun lock() {  
        val i = threadId()  
        val j = 1 - i  
        flag[i] = true  
        while (flag[j]) {}  
    }
```

"true" - has the lock (sort of)

```
    fun unlock() {  
        val i = threadId()  
        flag[i] = false  
    }
```

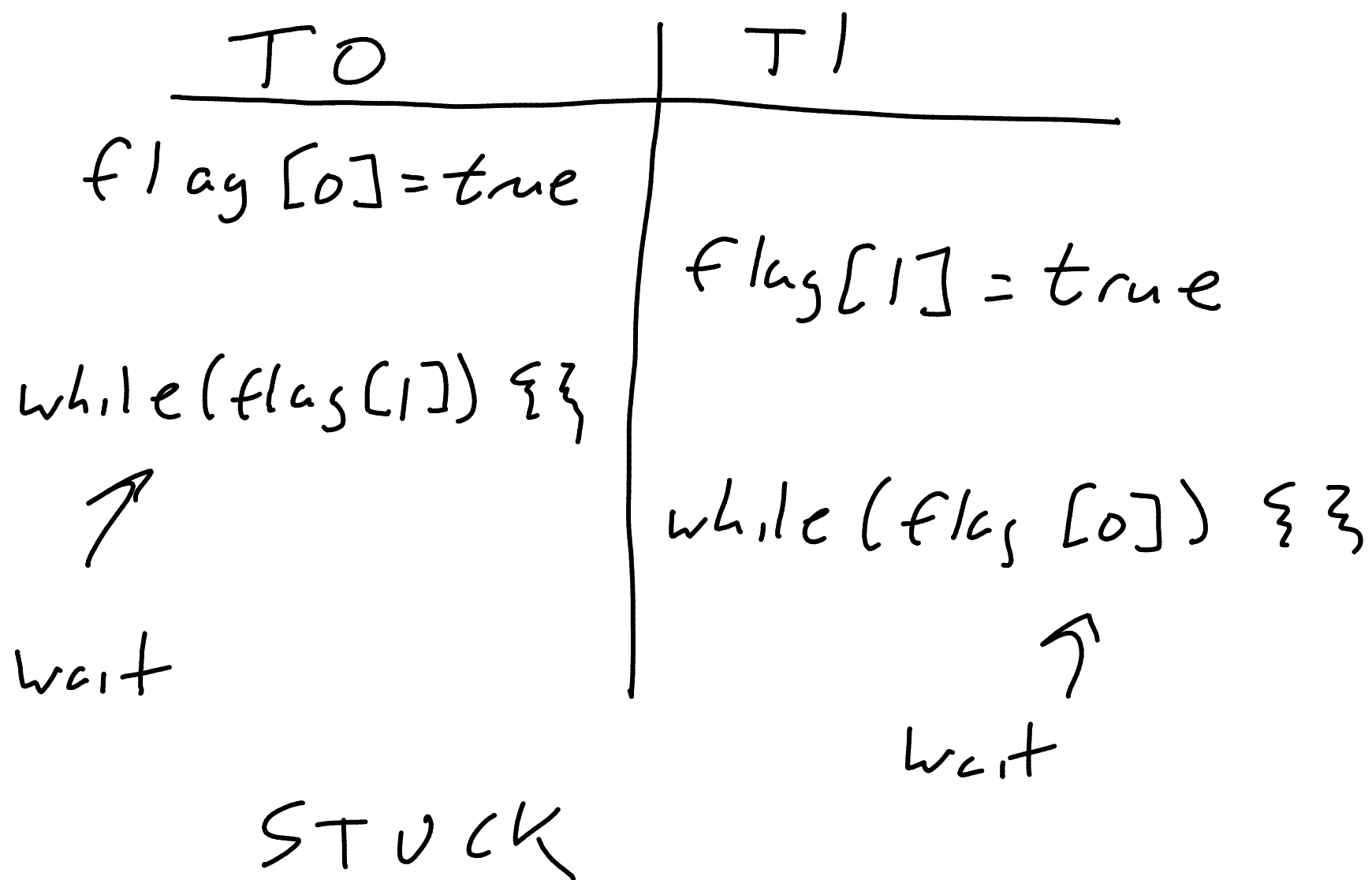
```
}
```

← As long as the other thread has the lock, wait

This does mutual exclusion. That's good!
How do you know that?

Prove it. [Wait, for now]

This does not enable deadlock freedom
- it happens with a bad interleaving on
lock()



```

class Lock2 {
    @Volatile var yielder = 0

    fun lock() {
        val i = threadId()
        yielder = i
        while (yielder == i) {}
    }

    fun unlock() {
    }
}

```

Also does mutual exclusion
(not obvious, but maybe can convince yourself)

This also has deadlock problems, but
when it doesn't interleave

